1. pgvector (already added to your models, not yet actively used)
A PostgreSQL extension that adds a vector data type and similarity search operators directly into Postgres. Instead of running a separate vector database (Pinecone, Weaviate, etc.), you store embeddings as a column in the same database as everything else and query them with near-normal SQL. Interview angle: be ready to explain why you chose pgvector over a dedicated vector DB — likely answer: simpler ops (one database, not two), good enough performance at small-to-medium scale, and it means your embeddings live right next to the relational data they belong to.
2. Gemini embeddings API
Google's API that turns text into a vector (a list of numbers representing meaning). This is what actually fills in the embedding field on ReferenceSnippet. Interview angle: know the general concept of an embedding model — similar meaning → similar vectors — and that you're calling this via an HTTP API rather than running a model locally.
3. Cosine similarity / vector search
The math operation used to compare two embeddings and score how "similar" they are. This is what pgvector accelerates. Interview angle: be able to say, in plain terms, "cosine similarity measures the angle between two vectors — closer to 1 means more similar meaning" — that one sentence answers a lot of interview questions.
4. Celery
A Python library for running tasks in the background, outside the request/response cycle. When someone uploads a PDF, you don't want them waiting 10+ seconds for extraction + chunking + embedding to finish before the HTTP response returns — Celery lets that work happen asynchronously while the API responds immediately with "upload received, processing." Interview angle: understand the core problem Celery solves — decoupling slow work from the request cycle — more than the library's specific syntax.
5. Redis
An in-memory data store. In this project it's used as Celery's "broker" — the middleman that holds the queue of pending tasks so Celery workers know what to pick up and run. Interview angle: know that Redis here isn't your main database, it's message-passing infrastructure for the task queue.
6. RAG (Retrieval-Augmented Generation) — the overall pattern, not a specific library
This is the umbrella concept tying steps 1-5 together: instead of an LLM answering purely from what it memorized in training, you retrieve relevant real content first (via vector search) and hand that to the LLM as grounding context before it generates a response. Interview angle: this is probably the single most valuable term to be fluent in — be ready to describe the full loop: document → chunk → embed → store → (later) retrieve by similarity → feed to LLM as context.
Already used, worth having ready too (since interviewers may ask about your stack broadly):

Django + DRF — the web framework and REST API toolkit
PostgreSQL — your relational database
Docker — containerizing the app for consistent setup
pypdf — PDF text extraction (already installed)